

HOSPITAL MANAGEMENT ERP

Developer Onboarding Guide

Step-by-Step Instructions for Setup, Understanding, and Development

Version 1.0 | April 2026 | Siance Software

Table of Contents

1. Project Overview

1.1 What Is This Application?

The Hospital Management ERP (HM-ERP) is a comprehensive, microservices-based platform that digitizes all hospital operations: patient registration, outpatient/inpatient management, diagnostics (lab, radiology), pharmacy, billing, inventory, HR, blood bank, dietary, quality management, and analytics.

1.2 Technology Stack

Technology	Version	Purpose
Java	21 (LTS)	Primary language with virtual threads
Spring Boot	3.3.x	Microservice framework
Spring Cloud	2023.0.3	Config, Discovery (Eureka), Gateway
PostgreSQL	16	Database per microservice
Apache Kafka	3.7+	Asynchronous event-driven communication
Redis	7	Caching, session management
Keycloak	24	Authentication, authorization, SSO
Elasticsearch	8	Full-text search (patient, drug search)
Docker + Docker Compose	25+	Containerization, local infra
Kubernetes	1.29+	Production orchestration
Flyway	10+	Database version control (migrations)
MapStruct	1.5.5	DTO/Entity mapping (zero-reflection)
Resilience4j	2.2	Circuit breaker, retry, rate limiter
Prometheus + Grafana	Latest	Metrics and dashboards
ELK Stack	8	Centralized logging
MinIO	Latest	Object storage (documents, images)

1.3 Architecture at a Glance

The system uses a microservices architecture with the following patterns:

- Database-per-Service: Each microservice has its own PostgreSQL database; no cross-DB queries.
- API Gateway: All client requests enter through hm-gateway (port 8080) which routes to services.
- Service Discovery: Eureka registry allows services to find each other by name (not hardcoded IPs).

- Centralized Config: hm-config-server provides configuration to all services from a central repository.
- Event-Driven: Services communicate asynchronously via Kafka events for state changes.
- RBAC Security: Keycloak issues JWT tokens; every service validates tokens and enforces role-based access.
- Shared Libraries: Common code (DTOs, exceptions, security, audit, Kafka) lives in shared Maven modules.

2. Project Structure

2.1 Top-Level Layout

hm-erp/	(Root Maven project)
pom.xml	(Parent BOM - dependency management)
docker-compose.yml	(Local infrastructure)
Makefile	(Developer commands)
Dockerfile	(Multi-stage build for all services)
.gitignore	
hm-common/	(Shared library: DTOs, exceptions, base entity)
hm-security/	(Shared library: Keycloak + Spring Security)
hm-kafka-common/	(Shared library: Kafka config, EventPublisher)
hm-audit/	(Shared library: Audit logging)
hm-config-server/	(Infrastructure: centralized config)
hm-discovery-server/	(Infrastructure: Eureka service registry)
hm-gateway/	(Infrastructure: API gateway + routing)
hm-patient-service/	(Business: patient registration, visits)
hm-appointment-service/	(Business: scheduling, tokens, queue)
hm-opd-service/	(Business: OPD consultation, prescriptions)
hm-billing-service/	(Business: invoicing, payments, refunds)
hm-master-data-service/	(Business: ICD codes, drug master, departments)
hm-notification-service/	(Business: SMS, email, WhatsApp notifications)
docker/	(Docker support files)
k8s/	(Kubernetes manifests)

2.2 Standard Service Internal Structure

Every business microservice follows this identical layout:

hm-patient-service/	
pom.xml	
src/main/java/com/siance/hm/patient/	
PatientServiceApplication.java	(Spring Boot entry point)
config/	(Swagger, Redis, custom beans)
controller/	(REST controllers - API layer)
service/	(Business logic layer)
repository/	(JPA repositories - data access)

entity/	(JPA entities - database tables)
dto/	(Request/Response DTOs)
mapper/	(MapStruct mappers: Entity <-> DTO)
event/	(Kafka event classes)
exception/	(Service-specific exceptions)
src/main/resources/	
application.yml	(Service configuration)
logback-spring.xml	(Logging configuration)
db/migration/	(Flyway SQL scripts)
src/test/	(Unit + integration tests)

3. Prerequisites and Environment Setup

3.1 Software Requirements

Software	Version	Installation
JDK	21+	https://adoptium.net/ (Eclipse Temurin recommended)
Maven	3.9+	https://maven.apache.org/download.cgi (or use <code>./mvnw</code>)
Docker Desktop	25+	https://www.docker.com/products/docker-desktop
Docker Compose	v2+	Included with Docker Desktop
Git	2.40+	https://git-scm.com/downloads
IDE	IntelliJ IDEA (recommended)	Ultimate or Community edition
Postman (optional)	Latest	API testing - https://www.postman.com

3.2 Verify Your Environment

Run these commands to confirm everything is installed:

```
java --version      # Should show 21.x.x
mvn --version       # Should show 3.9+
docker --version    # Should show 25+
docker compose version # Should show v2.x
git --version       # Should show 2.40+
```

3.3 IDE Setup (IntelliJ IDEA)

1. Open IntelliJ IDEA and select File > Open, navigate to hm-erp/ folder
2. Wait for Maven import to complete (may take 2-3 minutes first time)
3. Install plugins: Lombok, MapStruct Support, Spring Boot, Docker
4. Enable annotation processing: Settings > Build > Compiler > Annotation Processors > Enable
5. Set Project SDK to Java 21: File > Project Structure > Project > SDK
6. Verify: right-click pom.xml > Maven > Reload Project

4. Getting Started (First-Time Setup)

4.1 Clone the Repository

```
git clone https://github.com/siance/hm-erp.git
cd hm-erp
```

4.2 Start Infrastructure

This starts PostgreSQL, Kafka, Redis, Keycloak, Elasticsearch, MinIO, Prometheus, and Grafana:

```
make run-infra
# OR
docker-compose up -d
```

Wait about 30-60 seconds for all services to become healthy. Verify:

```
docker-compose ps    # All should show 'healthy' or 'running'
```

4.3 Access Infrastructure UIs

Service	URL	Credentials
Keycloak Admin	http://localhost:8180	admin / admin
Kafka UI	http://localhost:9080	(no auth)
Prometheus	http://localhost:9090	(no auth)
Grafana	http://localhost:3000	admin / admin
MinIO Console	http://localhost:9001	minio-admin / minio-secret-2026
Mailhog (email)	http://localhost:8025	(no auth)
Elasticsearch	http://localhost:9200	(no auth)

4.4 Build the Project

```
make build
# OR
mvn clean package -DskipTests
```

This compiles all 13 modules. First build downloads dependencies (~5 minutes). Subsequent builds take ~30 seconds.

4.5 Start Services (Development Order)

Services must start in this order due to dependencies:

1. Config Server (provides configuration to all services):

```
make run-config
# Wait until you see: 'Started ConfigServerApplication'
```

2. Discovery Server (service registry):

```
make run-discovery
# Wait until you see: 'Started DiscoveryServerApplication'
```

3. API Gateway:

```
make run-gateway
```

4. Business services (in any order):

```
make run-patient      # Patient Service (port 8100)
make run-appointment  # Appointment Service (port 8101)
make run-opd          # OPD Service (port 8102)
make run-billing       # Billing Service (port 8103)
make run-masterdata    # Master Data Service (port 8104)
make run-notification  # Notification Service (port 8105)
```

4.6 Verify Everything Works

Once services are running, test the patient service API:

Step 1: Get an access token from Keycloak:

```
curl -X POST http://localhost:8180/realms/hospital/protocol/openid-connect/token \
-H 'Content-Type: application/x-www-form-urlencoded' \
-d 'grant_type=password&client_id=hm-frontend&username=admin&password=admin123'
```

Step 2: Copy the access_token from the response and call the patient API:

```
curl -X POST http://localhost:8080/api/v1/patients \
-H 'Authorization: Bearer <YOUR_TOKEN>' \
-H 'Content-Type: application/json' \
-d '{
  "firstName": "Rajesh",
  "lastName": "Kumar",
  "gender": "MALE",
```

```
"dateOfBirth": "1985-03-15",  
"primaryPhone": "+919876543210",  
"email": "rajesh@example.com"  
}'
```

Step 3: Open Swagger UI for interactive API docs:

<http://localhost:8100/swagger-ui.html>

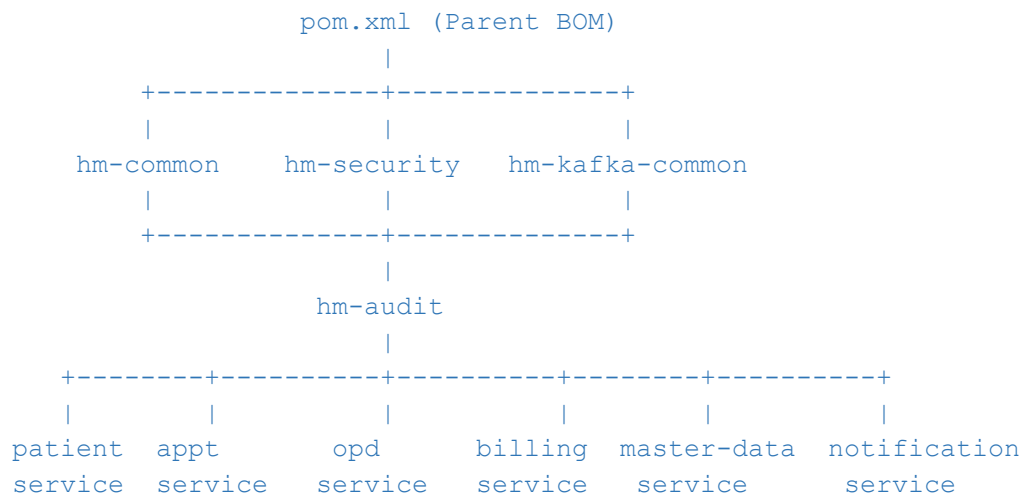
5. Understanding the Architecture

5.1 Request Flow (End-to-End)

Here is how a typical API request flows through the system:

1. Client sends HTTP request to API Gateway (port 8080)
2. Gateway validates JWT token with Keycloak (checks signature + expiry)
3. Gateway adds correlation ID header and routes to target service via Eureka lookup
4. Target service (e.g., patient-service) receives request at its controller
5. Controller validates input (@Valid), calls service layer
6. Service layer executes business logic, calls repository for DB operations
7. Repository uses JPA/Hibernate to query PostgreSQL
8. Service publishes Kafka event (e.g., patient.registered) for other services
9. Response flows back through gateway to client
10. Notification-service (Kafka consumer) picks up event and sends SMS/email

5.2 Module Dependency Hierarchy



5.3 How Services Communicate

Pattern	Example	When to Use
REST (sync)	Patient service calls Master Data service to validate ICD code	Need immediate response; data lookup
Kafka Event (async)	Patient registered -> Notification service sends welcome SMS	State change notification; no immediate response needed
Redis Cache	Patient profile cached for 15 min after first lookup	Frequently accessed, rarely changed data

Shared Database	NEVER! Each service has its own DB	This pattern is explicitly forbidden
-----------------	------------------------------------	--------------------------------------

5.4 Kafka Event Flow Example

When a new patient is registered:

1. PatientService.registerPatient() saves patient to patient_db
2. PatientService publishes PatientRegisteredEvent to topic 'hm.patient.registered'
3. NotificationService (consumer) receives event, sends welcome SMS
4. BillingService (consumer) receives event, creates patient billing profile
5. AnalyticsService (consumer) receives event, updates patient count metrics

6. Key Concepts for Developers

6.1 BaseEntity (All Entities Inherit This)

Every JPA entity extends BaseEntity from hm-common. This provides:

- UUID primary key (auto-generated, no sequence conflicts across services)
- created_at, updated_at (auto-populated by Hibernate)
- created_by, updated_by (auto-populated by AuditAware from JWT)
- deleted_at (soft delete - records are never physically removed)
- version (optimistic locking - prevents concurrent update conflicts)

6.2 API Response Format (Consistent Across All Services)

All APIs return the standard ApiResponse wrapper:

```
{
  "success": true,
  "message": "Created successfully",
  "data": { ... },           // The actual response payload
  "error": null,             // Only present on failure
  "timestamp": "2026-04-21T10:30:00Z"
}
```

Error responses:

```
{
  "success": false,
  "data": null,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Validation failed",
    "details": { "firstName": "must not be blank" }
  }
}
```

6.3 Security / RBAC

Security is handled by Keycloak + Spring Security:

- Keycloak issues JWT tokens containing user roles (ADMIN, DOCTOR, NURSE, etc.)
- hm-security library extracts roles from JWT and maps to Spring Security authorities
- Controllers use @PreAuthorize annotations to enforce role-based access
- Example: @PreAuthorize("hasAnyRole('ADMIN', 'DOCTOR')") on a method

6.4 Database Migrations (Flyway)

Schema changes are managed by Flyway migration scripts:

- Location: src/main/resources/db/migration/
- Naming: V{version}__{description}.sql (e.g., V1__create_patient_tables.sql)
- Migrations run automatically on service startup
- NEVER modify an existing migration after it has been applied - always create a new one
- Example for adding a column: V2__add_phone_verified_to_patients.sql

6.5 Caching (Redis)

Frequently accessed data is cached in Redis:

- @Cacheable("patients", key="#uhid") - caches method result
- @CacheEvict("patients", key="#uhid") - invalidates cache on update
- Cache TTL: 15 minutes for patients, 30 minutes default
- Always evict cache when data changes to avoid stale reads

6.6 Error Handling

All exceptions are handled globally by GlobalExceptionHandler in hm-common:

- ResourceNotFoundException -> 404 with NOT_FOUND code
- DuplicateResourceException -> 409 with DUPLICATE code
- BusinessException -> 422 with custom code
- MethodArgumentNotValidException -> 400 with field-level errors
- Any other Exception -> 500 with INTERNAL_ERROR (details hidden from client)

7. How to Add New Features

7.1 Adding a New API Endpoint

Example: Adding a 'merge patients' endpoint to patient-service:

1. Create DTO: `src/.../dto/PatientMergeDTO.java`
2. Add service method: `PatientService.mergePatients(MergeDTO dto)`
3. Add controller endpoint: `@PostMapping("/merge")` in `PatientController`
4. Write unit test in `PatientServiceTest`
5. Write integration test with `Testcontainers`
6. Update Swagger annotations (`@Operation`, `@ApiResponse`)

7.2 Adding a New Database Table

1. Create a new Flyway migration: `V2__create_consents_table.sql`
2. Create JPA entity: `Consent.java` extending `BaseEntity`
3. Create repository: `ConsentRepository` extends `JpaRepository`
4. Create DTOs: `ConsentCreateDTO`, `ConsentResponseDTO`
5. Create mapper: `ConsentMapper` (`MapStruct` interface)
6. Create service: `ConsentService` with business logic
7. Create controller: `ConsentController` with REST endpoints
8. Write tests

7.3 Adding a New Microservice

To create a new service (e.g., `hm-ipd-service` for Phase 2):

1. Copy `hm-patient-service` as template, rename to `hm-ipd-service`
2. Update `pom.xml`: change `artifactId`, `name`, `description`
3. Rename package from `com.siance.hm.patient` to `com.siance.hm.ipd`
4. Rename `Application` class to `IpServiceApplication`
5. Update `application.yml`: `port`, `database name`, `kafka group-id`
6. Create Flyway migration for the new database schema
7. Add module to root `pom.xml` `<modules>` section
8. Add route in `hm-gateway` `application.yml`
9. Add config in `hm-config-server/config-repo/hm-ipd-service.yml`
10. Add Prometheus scrape target in `docker/prometheus/prometheus.yml`

7.4 Publishing and Consuming Kafka Events

Publishing an event (in any service):

```
// 1. Create event class in event/ package
@Data @SuperBuilder @NoArgsConstructor
public class PatientAdmittedEvent extends BaseEvent {
    private String uhid;
    private String bedId;
    private String admittingDoctor;
}

// 2. Publish from service layer
PatientAdmittedEvent event = PatientAdmittedEvent.builder()...build();
event.initDefaults("PATIENT_ADMITTED", "hm-ipd-service");
eventPublisher.publish(KafkaTopics.IPD_PATIENT_ADMITTED, uhid, event);
```

Consuming an event (in another service):

```
@KafkaListener(topics = KafkaTopics.IPD_PATIENT_ADMITTED)
public void handlePatientAdmitted(PatientAdmittedEvent event) {
    log.info("Patient admitted: {}", event.getUhid());
    // Process event: create billing record, notify dietary, etc.
}
```

8. Testing

8.1 Running Tests

```
make test           # Run all unit tests
mvn test           # Same thing
mvn test -pl hm-patient-service # Test single module
```

8.2 Test Types

Type	Location	Tools	What It Tests
Unit Tests	src/test/.../service/	JUnit 5 + Mockito	Service logic in isolation (mocked dependencies)
Integration Tests	src/test/.../repository/	Testcontainers + Spring Boot Test	Real database queries against PostgreSQL container
Controller Tests	src/test/.../controller/	MockMvc + @WebMvcTest	HTTP request/response, validation, serialization
Kafka Tests	src/test/.../event/	EmbeddedKafka / Testcontainers	Event publishing and consumption

8.3 Writing a Unit Test

```
@ExtendWith(MockitoExtension.class)
class PatientServiceTest {
    @Mock private PatientRepository patientRepository;
    @Mock private PatientMapper patientMapper;
    @Mock private EventPublisher eventPublisher;
    @InjectMocks private PatientService patientService;

    @Test
    void registerPatient_ValidData_ReturnsPatient() {
        // Given
        PatientCreatedDTO dto = PatientCreatedDTO.builder()
            .firstName("Amit").lastName("Sharma").gender("MALE").build();
        when(patientRepository.getNextUhidSequence()).thenReturn(1L);
        when(patientRepository.save(any())).thenReturn(i ->
i.getArgument(0));
        when(patientMapper.toEntity(dto)).thenReturn(new Patient());
        when(patientMapper.toResponseDTO(any())).thenReturn(new
PatientResponseDTO());

        // When
        PatientResponseDTO result = patientService.registerPatient(dto);

        // Then
        assertNotNull(result);
    }
}
```

```
        verify(patientRepository).save(any());  
        verify(eventPublisher).publish(eq(KafkaTopics.PATIENT_REGISTERED),  
any(), any());  
    }  
}
```

9. Common Developer Commands

Command	What It Does
make run-infra	Start all infrastructure (Postgres, Kafka, Redis, Keycloak, etc.)
make stop-infra	Stop infrastructure
make reset-infra	Stop infrastructure AND delete all data (volumes)
make build	Build all modules (skip tests)
make test	Run all unit tests
make run-patient	Start patient service on port 8100
make run-gateway	Start API gateway on port 8080
docker-compose logs -f kafka	Follow Kafka logs
docker-compose exec postgres psql -U postgres patient_db	Connect to patient database
curl localhost:8100/actuator/health	Check patient service health
curl localhost:8761	Eureka dashboard (browser)
mvn -pl hm-patient-service flyway:info	Show migration status
mvn -pl hm-patient-service spring-boot:run -Dspring-boot.run.profiles=dev	Run with dev profile

10. Service Port Map

Service	Port	URL	Database
hm-config-server	8888	http://localhost:8888	Git-backed config
hm-discovery-server	8761	http://localhost:8761	In-memory
hm-gateway	8080	http://localhost:8080	None
hm-patient-service	8100	http://localhost:8100/swagger-ui.html	patient_db
hm-appointment-service	8101	http://localhost:8101/swagger-ui.html	appointment_db
hm-opd-service	8102	http://localhost:8102/swagger-ui.html	opd_db
hm-billing-service	8103	http://localhost:8103/swagger-ui.html	billing_db
hm-master-data-service	8104	http://localhost:8104/swagger-ui.html	master_db
hm-notification-service	8105	http://localhost:8105/swagger-ui.html	notification_db
Keycloak	8180	http://localhost:8180	keycloak_db
Kafka	9092	localhost:9092	N/A
Kafka UI	9080	http://localhost:9080	N/A
Redis	6379	localhost:6379	N/A
PostgreSQL	5432	localhost:5432	Multiple
Elasticsearch	9200	http://localhost:9200	N/A
Prometheus	9090	http://localhost:9090	N/A
Grafana	3000	http://localhost:3000	N/A
MinIO	9000/ 9001	http://localhost:9001	N/A
Mailhog	8025	http://localhost:8025	N/A

11. Troubleshooting

11.1 Common Issues

Problem: Service fails to start with 'Connection refused' to PostgreSQL

- Solution: Ensure docker-compose is running: `docker-compose ps`
- Check PostgreSQL is healthy: `docker-compose logs postgres`
- Database might not be created: `docker-compose exec postgres psql -U postgres -c "\l"`

Problem: 'Could not resolve placeholder' in application.yml

- Solution: Config server must be running first (port 8888)
- Or set `spring.config.import` to 'optional:' prefix to allow standalone startup

Problem: 401 Unauthorized on API calls

- Solution: Get a fresh token from Keycloak (tokens expire after 5 minutes by default)
- Ensure the user has the correct role for the endpoint
- Check that Keycloak realm 'hospital' exists: `http://localhost:8180/admin`

Problem: Kafka connection errors

- Solution: Ensure Kafka is running: `docker-compose logs kafka`
- Verify `KAFKA_SERVERS` env variable points to `localhost:9092`
- Check Kafka UI at `http://localhost:9080` for topic list

Problem: Flyway migration fails

- Solution: Check the SQL syntax in the migration file
- If you modified an already-applied migration, you must repair: `mvn flyway:repair`
- For fresh start: drop the database and recreate it

Problem: Maven build fails with 'Cannot resolve dependencies'

- Solution: Build shared libraries first: `mvn install -pl hm-common,hm-security,hm-kafka-common,hm-audit`
- Then build the full project: `mvn package -DskipTests`

11.2 Useful Debug Commands

```
# Check what's registered in Eureka
curl http://eureka-admin:eureka-secret-2026@localhost:8761/eureka/apps
```

```
# Check service health
curl localhost:8100/actuator/health | jq

# View all Kafka topics
docker-compose exec kafka kafka-topics --bootstrap-server localhost:9092
--list

# Read messages from a Kafka topic
docker-compose exec kafka kafka-console-consumer --bootstrap-server
localhost:9092 \
    --topic hm.patient.registered --from-beginning

# Connect to a database directly
docker-compose exec postgres psql -U postgres patient_db

# Check Redis cache
docker-compose exec redis redis-cli KEYS '*'
```

12. Coding Conventions and Best Practices

12.1 Code Style

- Use Lombok (@Data, @Builder, @RequiredArgsConstructor) to reduce boilerplate
- Use MapStruct for DTO/Entity mapping (never manual mapping in service layer)
- All REST endpoints return ApiResponse<T> wrapper
- Use constructor injection (@RequiredArgsConstructor) not @Autowired
- Service methods should be @Transactional (readOnly=true for queries)
- Log at appropriate levels: ERROR for failures, WARN for recoverable issues, INFO for business events, DEBUG for technical details

12.2 Git Workflow

- Branch from develop: feature/HM-123-add-patient-merge
- Commit messages: feat(patient): add patient merge endpoint
- Create PR to develop, require at least 1 review
- Squash merge to develop, create release branches for deployment

12.3 API Design Rules

- REST resource naming: plural nouns (/api/v1/patients, not /api/v1/patient)
- Use path params for identity: /api/v1/patients/{uhid}
- Use query params for filtering: /api/v1/patients?status=ACTIVE&name=Kumar
- HTTP methods: POST (create), GET (read), PUT (full update), PATCH (partial), DELETE (soft)
- Always validate input with Jakarta Bean Validation (@NotBlank, @Size, @Email, etc.)
- Always add @Operation and @Tag annotations for Swagger documentation

12.4 Database Rules

- All tables use UUID primary keys (gen_random_uuid())
- All tables have audit columns (created_at, updated_at, created_by, updated_by)
- Soft delete only: use deleted_at column, never DROP or DELETE
- Use JSONB for flexible/dynamic attributes (metadata, config)
- Always add indexes for foreign keys and commonly queried columns
- Use Flyway for ALL schema changes; never modify the database manually

END OF GUIDE

Hospital Management ERP - Developer Onboarding Guide v1.0

Siance Software | April 2026